

pandas 데이터 파악과 조작

분석할 데이터를 수집(확보)하면 데이터의 특징을 파악하고 다루기 쉽게 변형하는 작업을 수행해야 한다

#3. 데이터 정제와 변환

데이터 정제

- 누락 데이터(NaN) 처리
- 중복 데이터 처리

데이터 변환

- 데이터 표준화 : 단위 환산, 자료형 변환
- 데이터 정규화
- 범주형 데이터 처리 : 구간 분할, 더미변수
- 시계열 데이터 처리

```
In [1]: import numpy as np
import pandas as pd
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

1. 데이터 정제

- 누락데이터 처리
- 중복데이터 처리

1-1. 누락데이터(missing value) 처리

- 누락데이터 확인 : isnull(), notnull(), isna(), notna()
- 누락데이터 삭제 : dropna()
- 누락데이터 치환 : fillna(), bfill(), ffill()

1) 누락데이터 확인

- isna() 또는 isnull()
- 결측치인 경우 True 반환
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.isnull.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.isna.html>

```
In [3]: # !pip install matplotlib, seaborn
```

```
In [4]: import seaborn as sns
sns.__version__
```

```
Out[4]: '0.13.2'
```

```
In [5]: # seaborn의 샘플데이터셋 https://github.com/mwaskom/seaborn-data
titanic = sns.load_dataset('titanic')
titanic.head(3)
```

```
Out[5]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN



```
In [6]: titanic.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    str
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    str
8   class       891 non-null    category
9   who         891 non-null    str
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    str
13  alive       891 non-null    str
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

```
In [8]: # 누락(결측치)데이터 확인 : isna(), isnull()
titanic.isna().sum()
titanic.isnull().sum()
```

```
Out[8]: survived      0
pclass              0
sex                 0
age                177
sibsp              0
parch              0
fare               0
embarked           2
class              0
who                0
adult_male         0
deck              688
embark_town        2
alive              0
alone              0
dtype: int64
```

```
Out[8]: survived      0
        pclass        0
        sex           0
        age          177
        sibsp         0
        parch         0
        fare          0
        embarked      2
        class         0
        who           0
        adult_male     0
        deck          688
        embark_town    2
        alive          0
        alone          0
        dtype: int64
```

```
In [9]: # 결측치가 있는 컬럼명 추출
        cols = titanic.columns
        cols[titanic.isna().sum() > 0]
```

```
Out[9]: Index(['age', 'embarked', 'deck', 'embark_town'], dtype='str')
```

```
In [10]: # 컬럼명 정상데이터 개수 :notnull(), notna()
         titanic.notnull().sum()
```

```
Out[10]: survived      891
        pclass        891
        sex           891
        age           714
        sibsp         891
        parch         891
        fare          891
        embarked      889
        class         891
        who           891
        adult_male     891
        deck          203
        embark_town    889
        alive          891
        alone          891
        dtype: int64
```

2) 누락데이터 제거

dropna(*, axis=0, how='any', thresh=_NoDefault.no_default, subset=None, inplace=False)

- axis :
 - 0 or 'index' : 결측치를 포함하는 행들을 삭제
 - 1 or 'columns' : 결측치를 포함하는 열들을 삭제
- how :
 - 'any' : NaN이 하나라도 존재하면 삭제
 - 'all' : 모든 데이터가 NaN인 경우 삭제
- thresh : 지정된 개수 이상의 NaN이 있는 경우 삭제
- subset : 결측치 삭제할 컬럼 이름들 (리스트형식)
- inplace : True이면 기존 데이터 변환

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html>

NaN이 500개 이상인 열을 모두 삭제

```
In [11]: 500/891*100
```

```
Out[11]: 56.11672278338945
```

결측치가 있는 컬럼 : ['age', 'embarked', 'deck', 'embark_town']

```
In [13]: # 결측치 수 -> age:177개, embarked:2개, deck: 688개
titanic.dropna(axis=1).columns
```

```
Out[13]: Index(['survived', 'pclass', 'sex', 'sibsp', 'parch', 'fare', 'class', 'who',
               'adult_male', 'alive', 'alone'],
               dtype='str')
```

```
In [14]: # 결측치가 500개 이상인 컬럼만 삭제
titanic.dropna(axis=1, thresh=500).columns
```

```
Out[14]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
               'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',
               'alone'],
               dtype='str')
```

age 열에 나이 데이터가 없는 모든 행 삭제

```
In [15]: df = titanic.dropna(axis=0, subset='age', how='any')
df.info()
```

```
<class 'pandas.DataFrame'>
Index: 714 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    714 non-null    int64
1   pclass      714 non-null    int64
2   sex         714 non-null    str
3   age         714 non-null    float64
4   sibsp       714 non-null    int64
5   parch       714 non-null    int64
6   fare        714 non-null    float64
7   embarked    712 non-null    str
8   class       714 non-null    category
9   who         714 non-null    str
10  adult_male   714 non-null    bool
11  deck        184 non-null    category
12  embark_town  712 non-null    str
13  alive       714 non-null    str
14  alone       714 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 70.2 KB
```

dropna(how='all')

```
In [16]: data = pd.DataFrame([[1, 6.5, 3],
                             [1, np.nan, np.nan],
                             [np.nan, np.nan, np.nan],
                             [np.nan, 6.5, 3]])

data
```

```
Out[16]:
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

```
In [18]: data.dropna()
data.dropna(how='any')
data.dropna(how='all')
data.dropna(axis='columns')
data.dropna(axis='columns', how='all')
```

```
Out[18]:
```

	0	1	2
0	1.0	6.5	3.0

```
Out[18]:
```

	0	1	2
0	1.0	6.5	3.0

```
Out[18]:
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
3	NaN	6.5	3.0

```
Out[18]:  
0  
1  
2  
3
```

```
Out[18]:
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

3) 누락데이터 치환

- pandas 2.X버전까지

```
fillna(value=None, *, method=None, axis=None, inplace=False, limit=None,  
downcast=None)
```

- value : 결측치를 치환할 값(여러개인 경우: 딕셔너리)
- method : forward/backward fill 방식 {'backfill', 'bfill', 'pad', 'ffill', None}
 - pad / ffill: NaN이 있는 행의 직전(forward) 행의 값으로 대체
 - backfill / bfill: NaN이 있는 행 바로 다음(backward) 행의 값으로 대체
 - **pd.bfill(), pd.ffill()로 대체 사용할 것**

- pandas 3.x버전 이후 :

```
DataFrame.fillna(value, *, axis=None, inplace=False, limit=None) -> 3.x버전
```

- axis : 결측치 치환할 방향 {0 or 'index', 1 or 'columns'}
- inplace : bool, default False
- limit : 치환 회수 {int, default None}
 - method 인수가 설정된 경우 method치환을 적용할 최대 수
 - method 인수가 설정되지 않은 경우 컬럼별로 위에서 부터 지정한 회수만큼 치환
- downcast : dict, default is None
 - A dict of item->dtype of what to downcast if possible

평균값으로 치환

```
In [22]: mean_age = titanic['age'].mean()  
print(f'평균나이: {mean_age}')  
titanic.age.head(10)
```

평균나이: 29.69911764705882

```
Out[22]: 0    22.0
         1    38.0
         2    26.0
         3    35.0
         4    35.0
         5     NaN
         6    54.0
         7     2.0
         8    27.0
         9    14.0
         Name: age, dtype: float64
```

```
In [23]: titanic.age.fillna(value=mean_age).head(10)
```

```
Out[23]: 0    22.000000
         1    38.000000
         2    26.000000
         3    35.000000
         4    35.000000
         5    29.699118
         6    54.000000
         7     2.000000
         8    27.000000
         9    14.000000
         Name: age, dtype: float64
```

가장 많이 나오는 값으로 치환

```
In [26]: titanic.embark_town.value_counts()
```

```
Out[26]: embark_town
Southampton    644
Cherbourg      168
Queenstown     77
Name: count, dtype: int64
```

```
In [28]: titanic[titanic.embark_town.isna()]
```

```
Out[28]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
61	1	1	female	38.0	0	0	80.0	NaN	First	woman	False	B
829	1	1	female	62.0	0	0	80.0	NaN	First	woman	False	B



```
In [30]: most = titanic.embark_town.value_counts().idxmax()
         print(f'최대승선항구명: {most}')
```

최대승선항구명: Southampton

```
In [31]: titanic.embark_town.fillna(most).iloc[[69, 829]]
```

```
Out[31]: 69    Southampton
         829    Southampton
         Name: embark_town, dtype: str
```

이웃하는 데이터 값으로 치환

- 데이터 유사성
- fillna(method='ffill' | 'pad' | 'bfill' | 'backfill') -> 3.0버전이후에서 삭제된 기능
- pad(), backfill()은 2.x버전에서는 존재
- ffill(), bfill()
- 출처

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.fillna.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.ffill.html>
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.bfill.html>

```
In [32]: titanic.iloc[59:64]
titanic.iloc[827:832]
```

```
Out[32]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
59	0	3	male	11.0	5	2	46.9000	S	Third	child	False	NaN
60	0	3	male	22.0	0	0	7.2292	C	Third	man	True	NaN
61	1	1	female	38.0	0	0	80.0000	NaN	First	woman	False	B
62	0	1	male	45.0	1	0	83.4750	S	First	man	True	C
63	0	3	male	4.0	3	2	27.9000	S	Third	child	False	NaN

```
Out[32]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
827	1	2	male	1.00	0	2	37.0042	C	Second	child	False
828	1	3	male	NaN	0	0	7.7500	Q	Third	man	True
829	1	1	female	62.00	0	0	80.0000	NaN	First	woman	False
830	1	3	female	15.00	1	0	14.4542	C	Third	child	False
831	1	2	male	0.83	1	1	18.7500	S	Second	child	False

- 바로 앞의 값으로 치환 (method='ffill | pad' , ffill())

```
In [33]: titanic['emb_t'] = titanic.embark_town.ffill()
titanic.iloc[59:64]
titanic.iloc[827:832]
```

```
Out[33]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
59	0	3	male	11.0	5	2	46.9000	S	Third	child	False	NaN
60	0	3	male	22.0	0	0	7.2292	C	Third	man	True	NaN
61	1	1	female	38.0	0	0	80.0000	NaN	First	woman	False	B
62	0	1	male	45.0	1	0	83.4750	S	First	man	True	C
63	0	3	male	4.0	3	2	27.9000	S	Third	child	False	NaN

Out[33]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
827	1	2	male	1.00	0	2	37.0042	C	Second	child	False
828	1	3	male	NaN	0	0	7.7500	Q	Third	man	True
829	1	1	female	62.00	0	0	80.0000	NaN	First	woman	False
830	1	3	female	15.00	1	0	14.4542	C	Third	child	False
831	1	2	male	0.83	1	1	18.7500	S	Second	child	False

- 바로 뒤의 값으로 치환 (method='bfill | backfill', bfill())

```
In [34]: titanic['emb_t'] = titanic.embark_town.bfill()  
titanic.iloc[59:64]  
titanic.iloc[827:832]
```

Out[34]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
59	0	3	male	11.0	5	2	46.9000	S	Third	child	False	NaN
60	0	3	male	22.0	0	0	7.2292	C	Third	man	True	NaN
61	1	1	female	38.0	0	0	80.0000	NaN	First	woman	False	B
62	0	1	male	45.0	1	0	83.4750	S	First	man	True	C
63	0	3	male	4.0	3	2	27.9000	S	Third	child	False	NaN

Out[34]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
827	1	2	male	1.00	0	2	37.0042	C	Second	child	False
828	1	3	male	NaN	0	0	7.7500	Q	Third	man	True
829	1	1	female	62.00	0	0	80.0000	NaN	First	woman	False
830	1	3	female	15.00	1	0	14.4542	C	Third	child	False
831	1	2	male	0.83	1	1	18.7500	S	Second	child	False

컬럼별 결측치 치환 값이 다른 경우

- fillna()의 value 인수

```
In [35]: col = ['col1','col2','col3','col4','col5']  
row = ['row1','row2','row3','row4','row5','row6']  
na = np.nan  
data = [[na, 2, na, 4, na],  
        [ 6, 7, na, 9, na],  
        [11, na, na, 14, 15],  
        [na, 17, 3, na, 20],  
        [na, 22, na, na, 25],  
        [na, na, 5, 3, na]]  
df = pd.DataFrame(data, row, col)  
df
```

Out[35]:

	col1	col2	col3	col4	col5
row1	NaN	2.0	NaN	4.0	NaN
row2	6.0	7.0	NaN	9.0	NaN
row3	11.0	NaN	NaN	14.0	15.0
row4	NaN	17.0	3.0	NaN	20.0
row5	NaN	22.0	NaN	NaN	25.0
row6	NaN	NaN	5.0	3.0	NaN

```
In [36]: df.fillna(value={'col1':'A', 'col2':'B', 'col3':'C', 'col4':'D', 'col5':'E'})
```

Out[36]:

	col1	col2	col3	col4	col5
row1	A	2.0	C	4.0	E
row2	6.0	7.0	C	9.0	E
row3	11.0	B	C	14.0	15.0
row4	A	17.0	3.0	D	20.0
row5	A	22.0	C	D	25.0
row6	A	B	5.0	3.0	E

결측치 변경 수행횟수 제한

- fillna()의 limit 인수
- method 인수가 설정된 경우 method치환을 적용할 최대 수
- method 인수가 설정되지 않은 경우 컬럼별로 위에서 부터 지정한 회수만큼 치환

```
In [37]: df
df.fillna('A', limit=1)
```

Out[37]:

	col1	col2	col3	col4	col5
row1	NaN	2.0	NaN	4.0	NaN
row2	6.0	7.0	NaN	9.0	NaN
row3	11.0	NaN	NaN	14.0	15.0
row4	NaN	17.0	3.0	NaN	20.0
row5	NaN	22.0	NaN	NaN	25.0
row6	NaN	NaN	5.0	3.0	NaN

Out[37]:

	col1	col2	col3	col4	col5
row1	A	2.0	A	4.0	A
row2	6.0	7.0	NaN	9.0	NaN
row3	11.0	A	NaN	14.0	15.0
row4	NaN	17.0	3.0	A	20.0
row5	NaN	22.0	NaN	NaN	25.0
row6	NaN	NaN	5.0	3.0	NaN

In [38]: `df.fillna('A', limit=3)`

Out[38]:

	col1	col2	col3	col4	col5
row1	A	2.0	A	4.0	A
row2	6.0	7.0	A	9.0	A
row3	11.0	A	A	14.0	15.0
row4	A	17.0	3.0	A	20.0
row5	A	22.0	NaN	A	25.0
row6	NaN	A	5.0	3.0	A

In [40]: `df`
`df.fillna('A', axis='columns', limit=2)`

Out[40]:

	col1	col2	col3	col4	col5
row1	NaN	2.0	NaN	4.0	NaN
row2	6.0	7.0	NaN	9.0	NaN
row3	11.0	NaN	NaN	14.0	15.0
row4	NaN	17.0	3.0	NaN	20.0
row5	NaN	22.0	NaN	NaN	25.0
row6	NaN	NaN	5.0	3.0	NaN

Out[40]:

	col1	col2	col3	col4	col5
row1	A	2.0	A	4.0	NaN
row2	6.0	7.0	A	9.0	A
row3	11.0	A	A	14.0	15.0
row4	A	17.0	3.0	A	20.0
row5	A	22.0	A	NaN	25.0
row6	A	A	5.0	3.0	NaN

1-2. 중복데이터(duplicate data) 처리

- 중복데이터 확인 : duplicated()
- 중복데이터 제거 : drop_duplicates()

1) 중복데이터 확인

duplicated(subset=None, keep='first')

- keep : {'first','last', False}, default='first'
 - first : 위부터 검사
 - last : 마지막부터 검사
 - False : Mark all duplicates as True.
- subset : 특정 컬럼이나, 여러 컬럼조합 기준 중복 찾기
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.duplicated.html>

```
In [41]: df = pd.DataFrame({'c1': ['a', 'a', 'b', 'a', 'b'],
                             'c2': [1, 1, 1, 2, 2],
                             'c3': [1, 1, 2, 2, 2]})
df
```

```
Out[41]:
```

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
In [43]: # 데이터프레임의 관측값의 중복 확인
df.duplicated()
df[df.duplicated()]
```

```
Out[43]:
```

0	False
1	True
2	False
3	False
4	False

dtype: bool

```
Out[43]:
```

	c1	c2	c3
1	a	1	1

```
In [44]: # 컬럼별 중복값 확인
df.c2
df.c2.duplicated()
df.duplicated(subset='c2')
```

```
Out[44]:
```

0	1
1	1
2	1
3	2
4	2

Name: c2, dtype: int64

```
Out[44]: 0    False
         1     True
         2     True
         3    False
         4     True
         Name: c2, dtype: bool
```

```
Out[44]: 0    False
         1     True
         2     True
         3    False
         4     True
         dtype: bool
```

```
In [45]: df
df.duplicated(keep='last')
```

```
Out[45]:
```

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
Out[45]: 0     True
         1    False
         2    False
         3    False
         4    False
         dtype: bool
```

```
In [46]: df.duplicated(keep='first')
```

```
Out[46]: 0    False
         1     True
         2    False
         3    False
         4    False
         dtype: bool
```

```
In [47]: df.duplicated(keep=False)
```

```
Out[47]: 0     True
         1     True
         2    False
         3    False
         4    False
         dtype: bool
```

```
In [49]: df.c2
df.duplicated(subset='c2', keep='first')
df.duplicated(subset='c2', keep='last')
df.duplicated(subset='c2', keep=False)
```

```
Out[49]: 0     1
         1     1
         2     1
         3     2
         4     2
         Name: c2, dtype: int64
```

```
Out[49]: 0    False
         1     True
         2     True
         3    False
         4     True
         dtype: bool
```

```
Out[49]: 0     True
         1     True
         2    False
         3     True
         4    False
         dtype: bool
```

```
Out[49]: 0     True
         1     True
         2     True
         3     True
         4     True
         dtype: bool
```

```
In [50]: df
df.duplicated(subset=['c2', 'c3'], keep='first')
df.duplicated(subset=['c2', 'c3'], keep='last')
```

```
Out[50]:
```

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
Out[50]: 0    False
         1     True
         2    False
         3    False
         4     True
         dtype: bool
```

```
Out[50]: 0     True
         1    False
         2    False
         3     True
         4    False
         dtype: bool
```

2) 중복데이터 제거

- 함수형식

```
drop_duplicates**(subset=None, *, keep='first', inplace=False,
ignore_index=False)
```

- https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop_duplicates.html

```
In [51]: # 중복된 행(관측값) 전체 기준 삭제
df
df.drop_duplicates()
```

Out[51]:

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

Out[51]:

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

```
In [52]: df
df.drop_duplicates(subset=['c1','c3'])
```

Out[52]:

	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

Out[52]:

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2

```
In [53]: df.drop_duplicates(subset=['c1','c3'], keep='last')
```

Out[53]:

	c1	c2	c3
1	a	1	1
3	a	2	2
4	b	2	2

- 중복된 행(컬럼)기준 삭제시 어떤 것을 남길 것인가가 핵심
 - 정렬 + 중복제거
 - 최신데이터만 남기는 경우 -> timestamp 기준으로 정렬하여 keep='last'로 수행
 - 점수가 높은 행만 남기는 경우 -> 내림차순 정렬한 후 keep='first'로 수행
 - 중복 개수/분포 파악한 후 중복값 삭제 -> value_counts(), groupby().size()
 - 중복된 컬럼 표시(flag) 작성 -> 중복 그룹 식별 -> keep=False

연습문제

```
In [70]: df = pd.DataFrame({
    "user_id": [101, 102, 103, 101, 104, 105, 105, 106, 107, 107],
    "email": ["kim@a.com", "lee@a.com", "park@a.com", "KIM@A.COM ", None,
              "choi@a.com", "choi@a.com", "han@a.com", "yoon@a.com", "yoon@a.com"],
    "phone": ["010-1111-2222", "010-3333-4444", "010-5555-6666", "01011112222", "010-7777",
              "010-9999-0000", "010-9999-0000", "010-2222-3333", "010-4444-5555", "010-44",
    "score": [90, 85, 88, 92, 70, 60, 75, 95, 81, 81],
    "updated_at": [
        "2026-02-01", "2026-02-03", "2026-02-05", "2026-02-10", "2026-02-11",
        "2026-02-02", "2026-02-12", "2026-02-15", "2026-02-01", "2026-02-20"
    ]
})

df["updated_at"] = pd.to_datetime(df["updated_at"])
df
```

```
Out[70]:
```

	user_id	email	phone	score	updated_at
0	101	kim@a.com	010-1111-2222	90	2026-02-01
1	102	lee@a.com	010-3333-4444	85	2026-02-03
2	103	park@a.com	010-5555-6666	88	2026-02-05
3	101	KIM@A.COM	01011112222	92	2026-02-10
4	104	NaN	010-7777-8888	70	2026-02-11
5	105	choi@a.com	010-9999-0000	60	2026-02-02
6	105	choi@a.com	010-9999-0000	75	2026-02-12
7	106	han@a.com	010-2222-3333	95	2026-02-15
8	107	yoon@a.com	010-4444-5555	81	2026-02-01
9	107	yoon@a.com	010-4444-5555	81	2026-02-20

1. `user_id` 기준으로 중복된 행만 조회하기

```
In [ ]:
```

2. `email` 기준으로 중복된 데이터 그룹 전체 조회하기(첫번째 행도 포함)

```
In [ ]:
```

3. `user_id` 기준으로 중복을 제거하고, 첫번째 데이터만 남기기

```
In [ ]:
```

4. `user_id` 기준으로 중복을 제거하고 가장 마지막 데이터만 남기기

```
In [ ]:
```

5. `email` 기준으로 중복된 데이터는 전부 제거하고 유니크(unique)한 데이터만 남기기

In []:

6. `user_id` 기준으로 최신(updates_at) 데이터만 남기기

In []:

7. `email` 컬럼의 중복 여부를 표시하는 새로운 컬럼 `is_dup_email` 생성하기

In []:

8. `phone` 컬럼에서 숫자만 남기고 `phone_num` 컬럼 생성한 후 이 컬럼 기준으로 중복데이터 조회하기

In []:

9. `user_id` 별로 score가 가장 높은 데이터만 남기기

10. `email` 데이터를 정규화(공백 제거 + 소문자 변환)하여 `email_norm` 컬럼을 생성하기

In []:

2. 데이터 변환

- 데이터 변환 : 단위 환산, 자료형 변환, 값의 부분집합 변경, 값 치환
- 범주형 데이터 처리 : 구간 분할, 더미변수
- 데이터 정규화 및 표준화
- 시계열 데이터 처리

2-1. 데이터 변환

- 단위환산
- 자료형 변환
- 함수나 매핑을 이용한 데이터 일부 변형
- 값 치환하기

예제데이터. UCI 자동차 연비 데이터(Auto_MPG)

<https://archive.ics.uci.edu/ml/datasets/Auto+MPG>

데이터 변수 이름

- | | |
|------------------|-----------------------|
| 1. mpg: | continuous |
| 2. cylinders: | multi-valued discrete |
| 3. displacement: | continuous |
| 4. horsepower: | continuous |
| 5. weight: | continuous |
| 6. acceleration: | continuous |
| 7. model year: | multi-valued discrete |

8. origin: multi-valued discrete
9. car name: string (unique for each instance)

```
In [56]: auto = pd.read_csv('data/auto_mpg.csv', header=None)
auto.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column  Non-Null Count  Dtype
---  -
0    0      398 non-null      float64
1    1      398 non-null      int64
2    2      398 non-null      float64
3    3      398 non-null      str
4    4      398 non-null      float64
5    5      398 non-null      float64
6    6      398 non-null      int64
7    7      398 non-null      int64
8    8      398 non-null      str
dtypes: float64(4), int64(3), str(2)
memory usage: 28.1 KB
```

```
In [57]: names = '''
1. mpg:          continuous
2. cylinders:    multi-valued discrete
3. displacement: continuous
4. horsepower:   continuous
5. weight:       continuous
6. acceleration: continuous
7. model year:   multi-valued discrete
8. origin:       multi-valued discrete
9. car name:     string (unique for each instance)
'''
```

```
In [67]: colnames = [item.split(':')[0].split('.')[1].strip() for item in names.split('\n')][1:-1]
colnames
```

```
Out[67]: ['mpg',
'cylinders',
'displacement',
'horsepower',
'weight',
'acceleration',
'model year',
'origin',
'car name']
```

```
In [69]: auto.columns = colnames
auto.tail()
```

Out[69]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
393	27.0	4	140.0	86.00	2790.0	15.6	82	1	ford mustang gl
394	44.0	4	97.0	52.00	2130.0	24.6	82	2	vw pickup
395	32.0	4	135.0	84.00	2295.0	11.6	82	1	dodge rampage
396	28.0	4	120.0	79.00	2625.0	18.6	82	1	ford ranger
397	31.0	4	119.0	82.00	2720.0	19.4	82	1	chevy s- 10

1) 단위환산

'mpg' 컬럼의 단위 환산

- mile per gallon
- kilo meter per liter(km/l)로 환산
 - 1 mile = 1.60934km
 - 1 gallon = 3.78541L

In [70]:

```
auto['km/L'] = auto['mpg']*1.60934/3.78541
auto.head()
```

Out[70]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name	km,
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu	7.65257
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	buick skylark 320	6.37714
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	plymouth satellite	7.65257
3	16.0	8	304.0	150.0	3433.0	12.0	70	1	amc rebel sst	6.80228
4	17.0	8	302.0	140.0	3449.0	10.5	70	1	ford torino	7.22742

Series.round(decimals=0, *args, **kwargs) 메서드

In [73]:

```
auto['km/L'] = auto['km/L'].round(2)
auto.head()
```

Out[73]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name	km/L
0	18.0	8	307.0	130.0	3504.0	12.0	70	1	chevrolet chevelle malibu	7.65
1	15.0	8	350.0	165.0	3693.0	11.5	70	1	buick skylark 320	6.38
2	18.0	8	318.0	150.0	3436.0	11.0	70	1	plymouth satellite	7.65
3	16.0	8	304.0	150.0	3433.0	12.0	70	1	amc rebel sst	6.80
4	17.0	8	302.0	140.0	3449.0	10.5	70	1	ford torino	7.23

2) 자료형 변환

컬럼별 데이터 유형

In [74]: `auto.dtypes`

```
Out[74]: mpg          float64
cylinders      int64
displacement   float64
horsepower     str
weight         float64
acceleration   float64
model year     int64
origin         int64
car name       str
km/L           float64
dtype: object
```

'horsepower' 컬럼

- 자동차 마력
- 수치형이나 문자열로 저장되어 있음

In [75]: `auto.horsepower.unique()`

```
Out[75]: <StringArray>
['130.0', '165.0', '150.0', '140.0', '198.0', '220.0', '215.0', '225.0',
'190.0', '170.0', '160.0', '95.00', '97.00', '85.00', '88.00', '46.00',
'87.00', '90.00', '113.0', '200.0', '210.0', '193.0', '?', '100.0',
'105.0', '175.0', '153.0', '180.0', '110.0', '72.00', '86.00', '70.00',
'76.00', '65.00', '69.00', '60.00', '80.00', '54.00', '208.0', '155.0',
'112.0', '92.00', '145.0', '137.0', '158.0', '167.0', '94.00', '107.0',
'230.0', '49.00', '75.00', '91.00', '122.0', '67.00', '83.00', '78.00',
'52.00', '61.00', '93.00', '148.0', '129.0', '96.00', '71.00', '98.00',
'115.0', '53.00', '81.00', '79.00', '120.0', '152.0', '102.0', '108.0',
'68.00', '58.00', '149.0', '89.00', '63.00', '48.00', '66.00', '139.0',
'103.0', '125.0', '133.0', '138.0', '135.0', '142.0', '77.00', '62.00',
'132.0', '84.00', '64.00', '74.00', '116.0', '82.00']
Length: 94, dtype: str
```

- 'horsepower' 값 중 '?'을 NaN으로 치환

```
In [77]: auto['horsepower'] = auto.horsepower.replace('?', np.nan)
auto.horsepower.unique()
```

```
Out[77]: <StringArray>
['130.0', '165.0', '150.0', '140.0', '198.0', '220.0', '215.0', '225.0',
'190.0', '170.0', '160.0', '95.00', '97.00', '85.00', '88.00', '46.00',
'87.00', '90.00', '113.0', '200.0', '210.0', '193.0',      nan, '100.0',
'105.0', '175.0', '153.0', '180.0', '110.0', '72.00', '86.00', '70.00',
'76.00', '65.00', '69.00', '60.00', '80.00', '54.00', '208.0', '155.0',
'112.0', '92.00', '145.0', '137.0', '158.0', '167.0', '94.00', '107.0',
'230.0', '49.00', '75.00', '91.00', '122.0', '67.00', '83.00', '78.00',
'52.00', '61.00', '93.00', '148.0', '129.0', '96.00', '71.00', '98.00',
'115.0', '53.00', '81.00', '79.00', '120.0', '152.0', '102.0', '108.0',
'68.00', '58.00', '149.0', '89.00', '63.00', '48.00', '66.00', '139.0',
'103.0', '125.0', '133.0', '138.0', '135.0', '142.0', '77.00', '62.00',
'132.0', '84.00', '64.00', '74.00', '116.0', '82.00']
Length: 94, dtype: str
```

```
In [81]: auto[auto.horsepower.isna()]
```

```
Out[81]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name	km/L
32	25.0	4	98.0	NaN	2046.0	19.0	71	1	ford pinto	10.63
126	21.0	6	200.0	NaN	2875.0	17.0	74	1	ford maverick	8.93
330	40.9	4	85.0	NaN	1835.0	17.3	80	2	renault lecar deluxe	17.39
336	23.6	4	140.0	NaN	2905.0	14.3	80	1	ford mustang cobra	10.03
354	34.5	4	100.0	NaN	2320.0	15.8	81	2	renault 18i	14.67
374	23.0	4	151.0	NaN	3035.0	20.5	82	1	amc concord dl	9.78



- 'horsepower'의 문자열을 실수형으로 변환

```
In [82]: auto.horsepower = auto.horsepower.astype('float')
auto.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg             398 non-null   float64
1   cylinders       398 non-null   int64
2   displacement    398 non-null   float64
3   horsepower      392 non-null   float64
4   weight          398 non-null   float64
5   acceleration    398 non-null   float64
6   model year      398 non-null   int64
7   origin          398 non-null   int64
8   car name        398 non-null   str
9   km/L            398 non-null   float64
dtypes: float64(6), int64(3), str(1)
memory usage: 31.2 KB
```

```
In [83]: # horsepower 컬럼이 결측치인 데이터 삭제
auto.dropna(subset=['horsepower'], inplace=True)
auto.info()
```

```
<class 'pandas.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg             392 non-null   float64
1   cylinders       392 non-null   int64
2   displacement    392 non-null   float64
3   horsepower      392 non-null   float64
4   weight          392 non-null   float64
5   acceleration    392 non-null   float64
6   model year      392 non-null   int64
7   origin          392 non-null   int64
8   car name        392 non-null   str
9   km/L            392 non-null   float64
dtypes: float64(6), int64(3), str(1)
memory usage: 33.7 KB
```

```
In [84]: auto.horsepower.unique()
```

```
Out[84]: array([130., 165., 150., 140., 198., 220., 215., 225., 190., 170., 160.,
        95., 97., 85., 88., 46., 87., 90., 113., 200., 210., 193.,
        100., 105., 175., 153., 180., 110., 72., 86., 70., 76., 65.,
        69., 60., 80., 54., 208., 155., 112., 92., 145., 137., 158.,
        167., 94., 107., 230., 49., 75., 91., 122., 67., 83., 78.,
        52., 61., 93., 148., 129., 96., 71., 98., 115., 53., 81.,
        79., 120., 152., 102., 108., 68., 58., 149., 89., 63., 48.,
        66., 139., 103., 125., 133., 138., 135., 142., 77., 62., 132.,
        84., 64., 74., 116., 82.] )
```

'origin' 컬럼

- 정수형 데이터 1, 2, 3 : 국가이름 'USA', 'EU', 'JPN'을 의미함

```
In [85]: auto.dtypes
```

```
Out[85]: mpg          float64
cylinders          int64
displacement       float64
horsepower         float64
weight            float64
acceleration       float64
model year         int64
origin             int64
car name           str
km/L              float64
dtype: object
```

```
In [86]: auto.origin.unique()
```

```
Out[86]: array([1, 3, 2])
```

```
In [87]: auto.origin.value_counts()
```

```
Out[87]: origin
1      245
3       79
2       68
Name: count, dtype: int64
```

- 정수값을 국가 이름으로 치환

```
In [90]: auto.origin = auto.origin.replace({1:'USA', 2:'EU', 3:'JPN'})
auto.origin.unique()
auto.info()
```

```
Out[90]: array(['USA', 'JPN', 'EU'], dtype=object)
```

```
<class 'pandas.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg             392 non-null   float64
1   cylinders        392 non-null   int64
2   displacement     392 non-null   float64
3   horsepower       392 non-null   float64
4   weight           392 non-null   float64
5   acceleration     392 non-null   float64
6   model year       392 non-null   int64
7   origin           392 non-null   object
8   car name         392 non-null   str
9   km/L            392 non-null   float64
dtypes: float64(6), int64(2), object(1), str(1)
memory usage: 33.7+ KB
```

- 범주형(category) 데이터로 변환

```
In [92]: auto.origin = auto.origin.astype('category')
auto.info()
```

```

<class 'pandas.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   mpg             392 non-null    float64
 1   cylinders        392 non-null    int64
 2   displacement     392 non-null    float64
 3   horsepower       392 non-null    float64
 4   weight           392 non-null    float64
 5   acceleration     392 non-null    float64
 6   model year       392 non-null    int64
 7   origin           392 non-null    category
 8   car name         392 non-null    str
 9   km/L             392 non-null    float64
dtypes: category(1), float64(6), int64(2), str(1)
memory usage: 31.0 KB

```

```

In [93]: auto.origin.cat.categories
auto.origin.cat.codes

```

```

Out[93]: Index(['EU', 'JPN', 'USA'], dtype='str')

```

```

Out[93]: 0      2
         1      2
         2      2
         3      2
         4      2
         ..
        393     2
        394     0
        395     2
        396     2
        397     2
Length: 392, dtype: int8

```

'model_year' 컬럼

: 모델 출시 연도

- sample() 메서드로 무작위로 3개 행을 선택하여 출력

```

In [95]: auto['model year'].sample(5)

```

```

Out[95]: 190      76
         186      76
         196      76
         376      82
          94      73
Name: model year, dtype: int64

```

```

In [96]: auto['model year'].value_counts()

```



```
Out[96]: model year
       73      40
       78      36
       76      34
       75      30
       82      30
       70      29
       79      29
       72      28
       77      28
       81      28
       71      27
       80      27
       74      26
Name: count, dtype: int64
```

```
In [97]: auto['model year'].unique()
```

```
Out[97]: array([70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82])
```

- 범주형 데이터로 변환

```
In [99]: auto['model year'] = auto['model year'].astype('category')
auto['model year']
```

```
Out[99]: 0      70
         1      70
         2      70
         3      70
         4      70
         ..
        393     82
        394     82
        395     82
        396     82
        397     82
Name: model year, Length: 392, dtype: category
Categories (13, int64): [70, 71, 72, 73, ..., 79, 80, 81, 82]
```

```
In [101... auto['model year'].cat.categories
```

```
Out[101... Index([70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82], dtype='int64')
```

```
In [102... # 순서형으로 변경
auto['model year'].cat.as_ordered()
```

```
Out[102... 0      70
         1      70
         2      70
         3      70
         4      70
         ..
        393     82
        394     82
        395     82
        396     82
        397     82
Name: model year, Length: 392, dtype: category
Categories (13, int64): [70 < 71 < 72 < 73 ... 79 < 80 < 81 < 82]
```

3) 함수나 매핑을 이용한 데이터 변형 : DataFrame.map()

: 데이터프레임의 열이나 시리즈, 배열 내의 값을 기반으로 데이터의 형태를 변형하려는 경우

=> map() : 한 객체 안에서 값의 부분집합을 변경

In [103...

```
data = pd.DataFrame({'food': ['베이컨', '소시지', '베이컨', '살라미',  
                              '콘비프', '베이컨', '살라미', '햄', '치즈'],  
                    'amount': [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})  
data
```

Out[103...

	food	amount
0	베이컨	4.0
1	소시지	3.0
2	베이컨	12.0
3	살라미	6.0
4	콘비프	7.5
5	베이컨	8.0
6	살라미	3.0
7	햄	5.0
8	치즈	6.0

- 음식의 원재료를 담은 딕셔너리 데이터를 열로 추가하려는 경우

```
animal = {'베이컨': '돼지', '소시지': '돼지', '살라미': '돼지', '콘비프': '소', '햄': '소', '치즈': '소'}
```

In [104...

```
animal = {'베이컨': '돼지', '소시지': '돼지', '살라미': '돼지', '콘비프': '소', '햄': '소', '치즈': '소'}  
data['animal'] = data['food'].map(animal)  
data
```

Out[104...

	food	amount	animal
0	베이컨	4.0	돼지
1	소시지	3.0	돼지
2	베이컨	12.0	돼지
3	살라미	6.0	돼지
4	콘비프	7.5	소
5	베이컨	8.0	돼지
6	살라미	3.0	돼지
7	햄	5.0	소
8	치즈	6.0	소

In [106...

```
def get_animal(x):  
    return animal[x]  
get_animal('소시지')  
data['food'].map(get_animal)
```

Out[106...

```
'돼지'
```

```
Out[106...]
0   돼지
1   돼지
2   돼지
3   돼지
4   소
5   돼지
6   돼지
7   소
8   소
Name: food, dtype: str
```

4) 값 치환하기 : Series.replace()

```
In [107...] data['animal2'] = data['food'].replace(animal)
```

```
In [108...] data
```

```
Out[108...]
   food  amount  animal  animal2
0  베이컨     4.0   돼지   돼지
1  소시지     3.0   돼지   돼지
2  베이컨    12.0   돼지   돼지
3  살라미     6.0   돼지   돼지
4  콘비프     7.5    소    소
5  베이컨     8.0   돼지   돼지
6  살라미     3.0   돼지   돼지
7    햄      5.0    소    소
8   치즈     6.0    소    소
```

- 누락된 데이터를 나타내는 -999를 NaN으로 치환하기

```
In [111...] s = pd.Series([1, -999, 2, -999, -1000, 3])
s
```

```
Out[111...]
0      1
1   -999
2      2
3   -999
4  -1000
5      3
dtype: int64
```

- -999와 -1000 모두 NaN으로 치환하기

```
In [110...] s.replace(-999, np.nan)
s
```

```
Out[110...]
0      1.0
1      NaN
2      2.0
3      NaN
4  -1000.0
5      3.0
dtype: float64
```

```
Out[110...] 0      1
            1      -999
            2       2
            3     -999
            4   -1000
            5       3
dtype: int64
```

- -999는 NaN으로, -1000은 0으로 치환하기

```
In [112...] s.replace([-999, -1000], np.nan)
```

```
Out[112...] 0      1.0
            1     NaN
            2      2.0
            3     NaN
            4     NaN
            5      3.0
dtype: float64
```

```
In [114...] s
s.replace({-999:np.nan, -1000:0})
```

```
Out[114...] 0      1
            1     -999
            2       2
            3     -999
            4   -1000
            5       3
dtype: int64
```

```
Out[114...] 0      1.0
            1     NaN
            2      2.0
            3     NaN
            4      0.0
            5      3.0
dtype: float64
```

2-2. 범주형 데이터 처리

1) 구간 분할(binning)

예제데이터. UCI 자동차 연비 데이터(Auto_MPG)

<https://archive.ics.uci.edu/ml/datasets/Auto+MPG>

```
In [115...] auto.tail()
```

Out[115...

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name	km/L
393	27.0	4	140.0	86.0	2790.0	15.6	82	USA	ford mustang gl	11.48
394	44.0	4	97.0	52.0	2130.0	24.6	82	EU	vw pickup	18.71
395	32.0	4	135.0	84.0	2295.0	11.6	82	USA	dodge rampage	13.60
396	28.0	4	120.0	79.0	2625.0	18.6	82	USA	ford ranger	11.90
397	31.0	4	119.0	82.0	2720.0	19.4	82	USA	chevy s-10	13.18

In [116...

```
auto.info()
```

```
<class 'pandas.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg              392 non-null    float64
1   cylinders        392 non-null    int64
2   displacement     392 non-null    float64
3   horsepower       392 non-null    float64
4   weight           392 non-null    float64
5   acceleration     392 non-null    float64
6   model year      392 non-null    category
7   origin           392 non-null    category
8   car name        392 non-null    str
9   km/L            392 non-null    float64
dtypes: category(2), float64(6), int64(1), str(1)
memory usage: 28.5 KB
```

'horsepower' 컬럼을 범주형 변수로 변환

: **cut()** 을 이용하여 '저출력', '보통출력', '고출력' 3구간으로 **구간화(binning)**

<https://pandas.pydata.org/docs/reference/api/pandas.cut.html?highlight=cut#pandas.cut>

In [117...

```
auto.horsepower.describe()
```

Out[117...

```
count    392.000000
mean     104.469388
std       38.491160
min       46.000000
25%       75.000000
50%       93.500000
75%      126.000000
max      230.000000
Name: horsepower, dtype: float64
```

In [118...

```
# 구간 경계 : ( 46. , 107.33333333], (107.33333333, 168.66666667]. (168.66666667, 230.]
pd.cut(auto.horsepower, bins=3, labels=['저출력', '보통출력', '고출력'])
```

```
Out[118...] 0    보통출력
            1    보통출력
            2    보통출력
            3    보통출력
            4    보통출력
            ...
            393   저출력
            394   저출력
            395   저출력
            396   저출력
            397   저출력
Name: horsepower, Length: 392, dtype: category
Categories (3, str): ['저출력' < '보통출력' < '고출력']
```

- np.histogram()을 이용하여 구간 경계값을 반환받아 사용

<https://numpy.org/doc/stable/reference/generated/numpy.histogram.html>

```
In [119...] freq, bins = np.histogram(auto.horsepower, bins=3)
print(f'freq:{freq}')
print(f'bins:{bins}')
```

```
freq:[257 103  32]
bins:[ 46.          107.33333333 168.66666667 230.          ]
```

```
In [120...] auto['hp_cat'] = pd.cut(auto.horsepower, bins=3, labels=['저출력','보통출력','고출력'])
auto[['horsepower','hp_cat']]
```

```
Out[120...]      horsepower  hp_cat
0          130.0  보통출력
1          165.0  보통출력
2          150.0  보통출력
3          150.0  보통출력
4          140.0  보통출력
...          ...      ...
393         86.0   저출력
394         52.0   저출력
395         84.0   저출력
396         79.0   저출력
397         82.0   저출력
```

392 rows × 2 columns

2) 더미(dummy) 변수

- 범주형 데이터를 회귀분석의 입력변수로 사용할 경우 범주형 변수의 데이터 값들을 0 또는 1의 값을 갖는 더미변수로 변환해주어야 함
- 크고 작음 보다는 어떤 특성이 있는지 없는지 여부만 표시
- 원핫인코딩(one-hot encoding)
- pd.get_dummies() 함수

https://pandas.pydata.org/docs/reference/api/pandas.get_dummies.html?highlight=get_dummies#

'hp_cat' 컬럼을 더미변수로 변환

```
In [121... hp_dummies = pd.get_dummies(auto['hp_cat'], dtype=int)
hp_dummies
```

```
Out[121...      저출력  보통출력  고출력
0         0         1         0
1         0         1         0
2         0         1         0
3         0         1         0
4         0         1         0
...      ...      ...      ...
393        1         0         0
394        1         0         0
395        1         0         0
396        1         0         0
397        1         0         0
```

392 rows × 3 columns

```
In [122... auto.head()
```

```
Out[122...      mpg  cylinders  displacement  horsepower  weight  acceleration  model
year  origin      car name  km/L
0    18.0         8        307.0         130.0   3504.0          12.0    70
USA  chevrolet
      chevelle
      malibu    7.65
1    15.0         8        350.0         165.0   3693.0          11.5    70
USA  buick
      skylark
      320      6.38
2    18.0         8        318.0         150.0   3436.0          11.0    70
USA  plymouth
      satellite  7.65
3    16.0         8        304.0         150.0   3433.0          12.0    70
USA  amc
      rebel sst  6.80
4    17.0         8        302.0         140.0   3449.0          10.5    70
USA  ford
      torino    7.23
```



```
In [123... pd.concat([auto, hp_dummies], axis=1)
```

Out[123...

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name	km/h
0	18.0	8	307.0	130.0	3504.0	12.0	70	USA	chevrolet chevelle malibu	7.69
1	15.0	8	350.0	165.0	3693.0	11.5	70	USA	buick skylark 320	6.38
2	18.0	8	318.0	150.0	3436.0	11.0	70	USA	plymouth satellite	7.69
3	16.0	8	304.0	150.0	3433.0	12.0	70	USA	amc rebel sst	6.80
4	17.0	8	302.0	140.0	3449.0	10.5	70	USA	ford torino	7.23
...	...	...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86.0	2790.0	15.6	82	USA	ford mustang gl	11.48
394	44.0	4	97.0	52.0	2130.0	24.6	82	EU	vw pickup	18.71
395	32.0	4	135.0	84.0	2295.0	11.6	82	USA	dodge rampage	13.60
396	28.0	4	120.0	79.0	2625.0	18.6	82	USA	ford ranger	11.90
397	31.0	4	119.0	82.0	2720.0	19.4	82	USA	chevy s- 10	13.18

392 rows × 14 columns



2-3. 정규화(normalization)

- scaling
 - 정규화 : min-max scaling
 - 표준화 : Z-score
- 숫자 데이터의 상대적인 크기 차이를 제거하기 위해, 데이터 값을 동일한 크기 기준으로 나눈 비율로 나타낸 변환
- 특성 벡터의 길이는 상관 없고 데이터의 방향만이 중요할 때 많이 사용

데이터를 최대값(의 절대값)으로 나눔

In [125... `auto.horsepower.dtype`

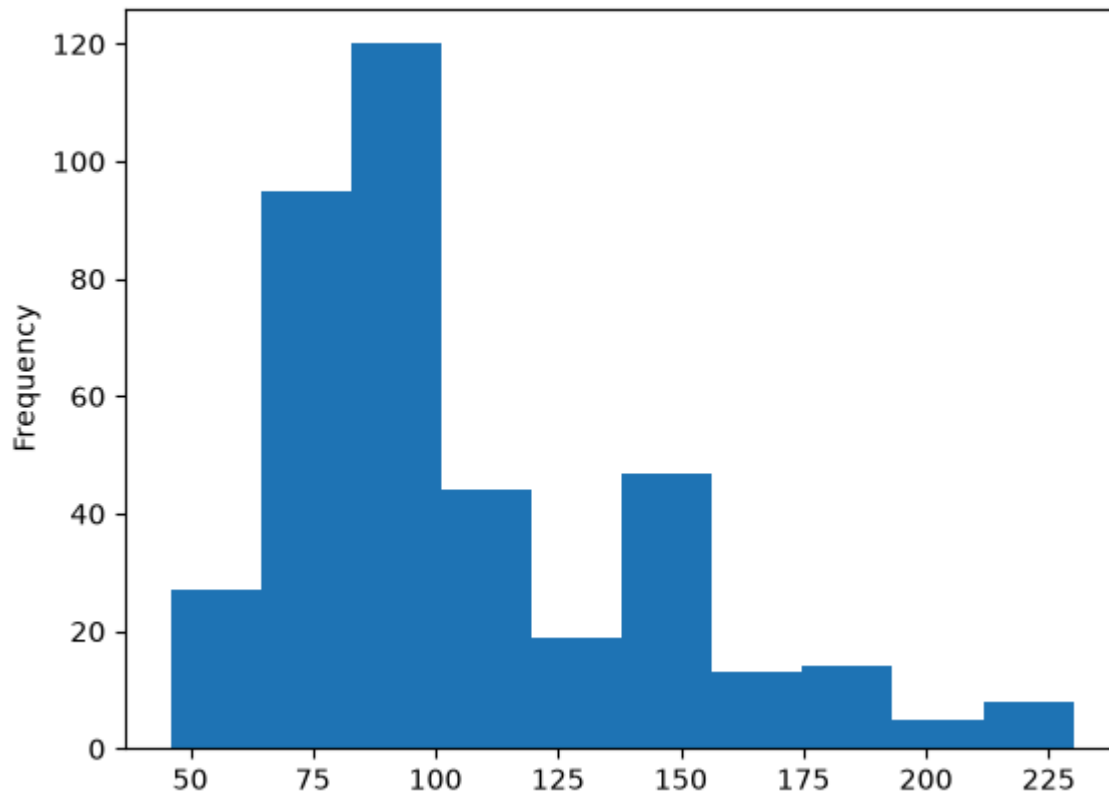
Out[125... `dtype('float64')`

In [126... `auto.horsepower.describe()`


```
Out[126... count    392.000000
mean     104.469388
std      38.491160
min      46.000000
25%      75.000000
50%      93.500000
75%     126.000000
max     230.000000
Name: horsepower, dtype: float64
```

```
In [127... auto.horsepower.plot.hist()
```

```
Out[127... <Axes: ylabel='Frequency'>
```

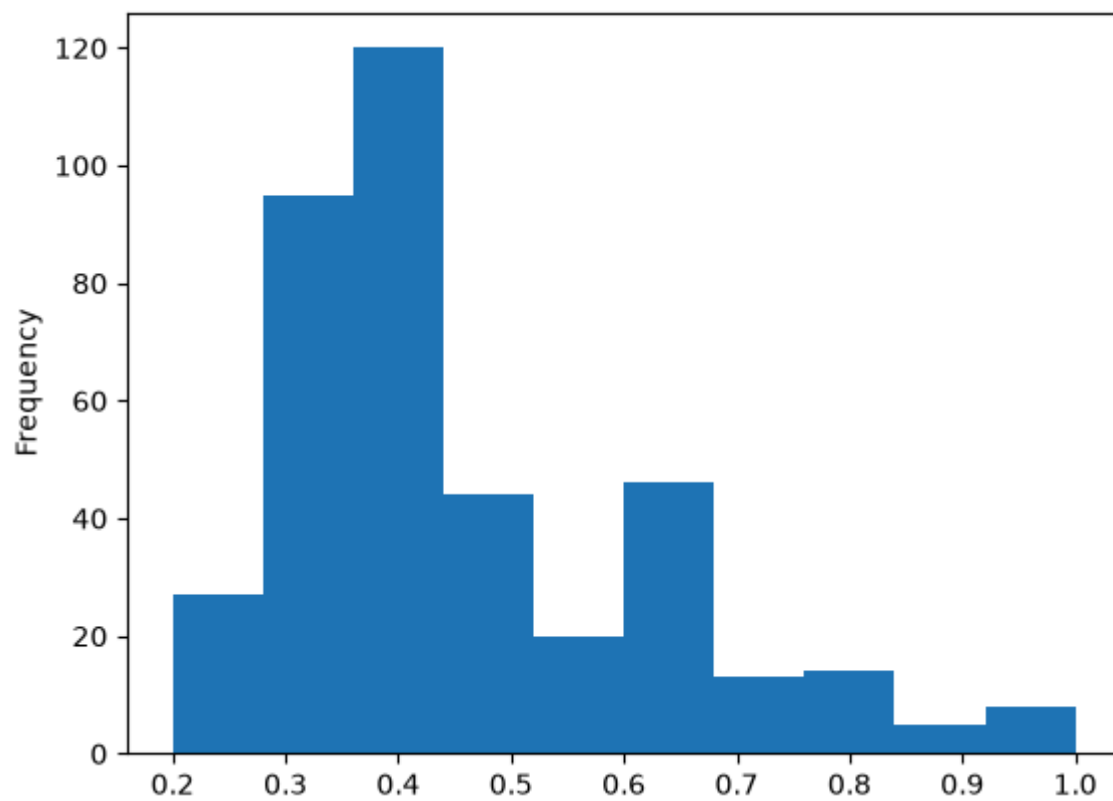


```
In [128... # x /abs(max(x))
hp1 = auto.horsepower/abs(auto.horsepower.max())
hp1.describe()
```

```
Out[128... count    392.000000
mean      0.454215
std       0.167353
min       0.200000
25%       0.326087
50%       0.406522
75%       0.547826
max       1.000000
Name: horsepower, dtype: float64
```

```
In [130... hp1.plot.hist()
```

```
Out[130... <Axes: ylabel='Frequency'>
```



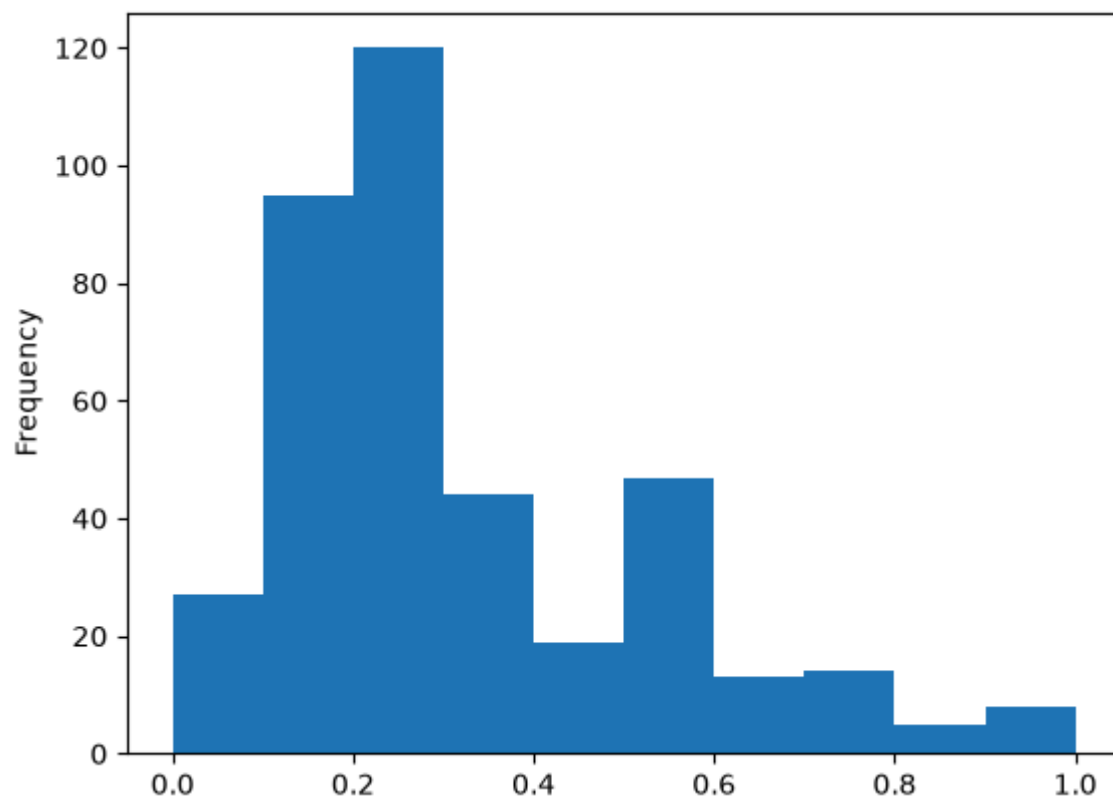
min-max scaling : 최소값은 0, 최대값은 1이 되도록 변환

```
In [133... min_max = auto.horsepower.max() - auto.horsepower.min()
min_x = auto.horsepower - auto.horsepower.min()
hp2 = min_x / min_max
hp2.describe()
```

```
Out[133... count    392.000000
mean       0.317768
std        0.209191
min        0.000000
25%        0.157609
50%        0.258152
75%        0.434783
max        1.000000
Name: horsepower, dtype: float64
```

```
In [134... hp2.plot.hist()
```

```
Out[134... <Axes: ylabel='Frequency'>
```



표준화(Standardization)

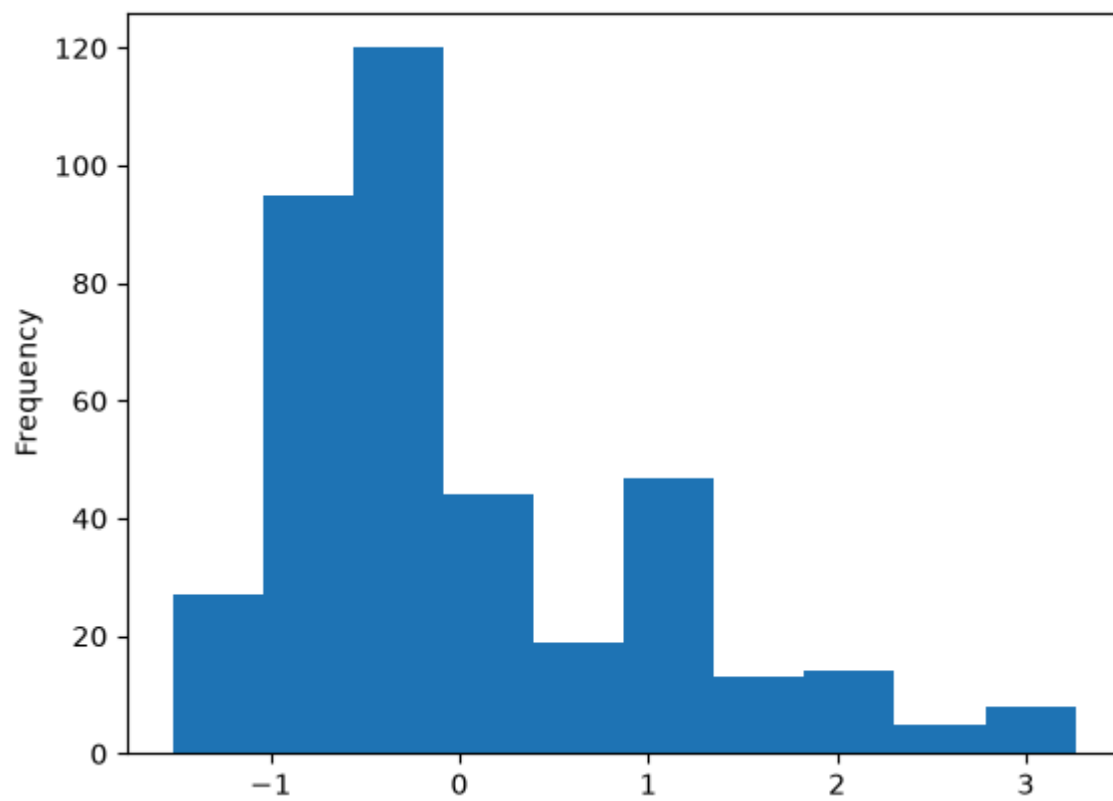
- Z-score 정규화
- 평균은 0, 표준편차는 1이 되도록 변환
- 이상치 파악

```
In [135... hp3 = (auto.horsepower - auto.horsepower.mean()) / auto.horsepower.std()  
hp3.describe()
```

```
Out[135... count    3.920000e+02  
mean      -1.631348e-16  
std        1.000000e+00  
min       -1.519034e+00  
25%       -7.656144e-01  
50%       -2.849846e-01  
75%        5.593651e-01  
max        3.261284e+00  
Name: horsepower, dtype: float64
```

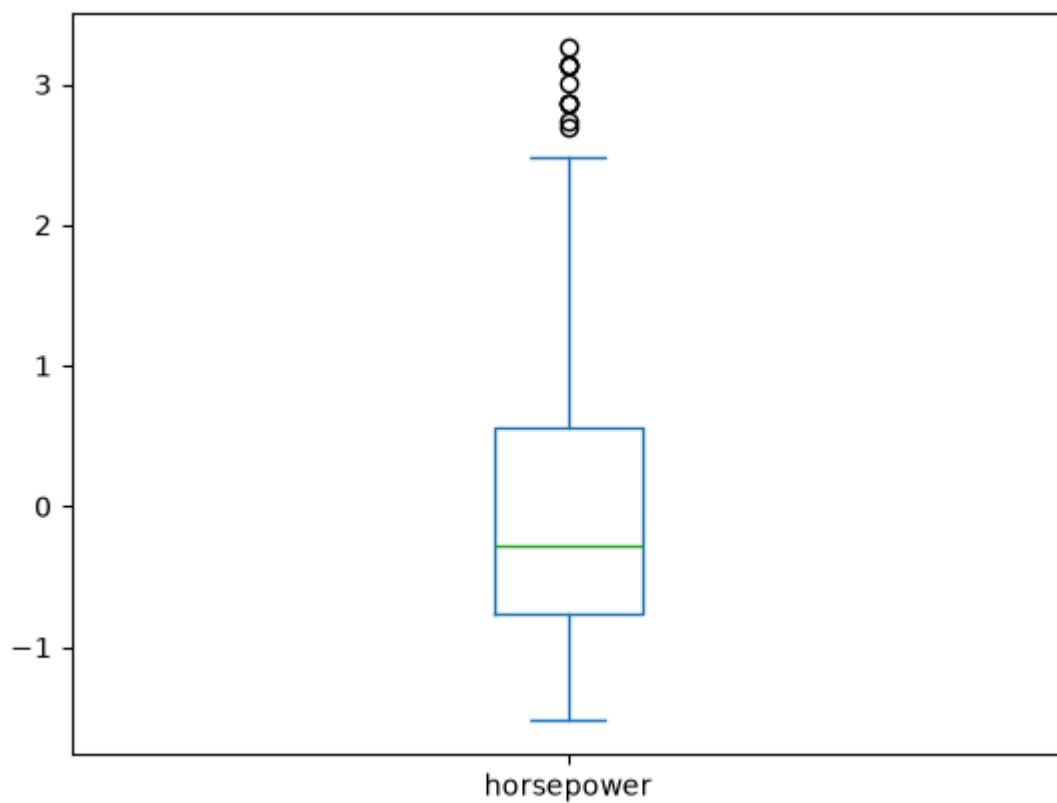
```
In [136... hp3.plot.hist()
```

```
Out[136... <Axes: ylabel='Frequency'>
```



In [137... `hp3.plot.box()`

Out[137... `<Axes: >`

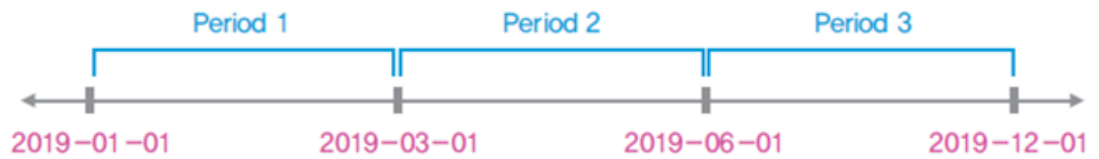


2-4. 시계열 데이터 처리

- 시계열 데이터를 데이터프레임의 행 인덱스로 사용하면 시간으로 기록된 데이터 분석에 매우 용이함
- Timestamp : 특정한 시점을 기록
- Period : 두 시점 사이의 일정한 기간



(a) 타임스탬프(Timestamp)



(b) 피리어드(Period)

- 다른 자료형을 시계열 객체로 변환
 - 문자열을 Timestamp로 변환 : `to_datetime()`
 - Timestamp를 Period로 변환 : `to_period()`
- 시계열 데이터 생성
 - Timestamp 배열 : `date_range()`
 - Period 배열 : `period_range()`
- 시계열 데이터 활용 : 날짜 데이터 분리, 날짜 인덱스 활용

https://pandas.pydata.org/docs/user_guide/timeseries.html

1) 다른 자료형을 시계열 객체로 변환

① 문자열을 Timestamp로 변환 : `pd.to_datetime()`

`to_datetime(arg, errors='raise', dayfirst=False, yearfirst=False, utc=None, format=None, exact=True, unit=None, infer_datetime_format=False, origin='unix', cache=True)`

- `datetime64` 자료형으로 변환

https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html

예제. 주식 거래 데이터(Netflix)

In [138...

```
stock = pd.read_csv('data/NFLX.csv')
stock.head()
```

Out[138...

	Date	Open	High	Low	Close	Adj Close	Volume
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200

```
In [139... stock.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4874 entries, 0 to 4873
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        4874 non-null   str
1   Open        4874 non-null   float64
2   High        4874 non-null   float64
3   Low         4874 non-null   float64
4   Close       4874 non-null   float64
5   Adj Close   4874 non-null   float64
6   Volume      4874 non-null   int64
dtypes: float64(5), int64(1), str(1)
memory usage: 266.7 KB
```

- Date 컬럼 : 문자열(object, str)을 Timestamp 객체로 변환

```
In [140... stock['new_date'] = pd.to_datetime(stock.Date)
stock.head()
stock.info()
```

Out[140...

	Date	Open	High	Low	Close	Adj Close	Volume	new_date
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002-05-23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002-05-24
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002-05-28
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002-05-29
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002-05-30

```
<class 'pandas.DataFrame'>
RangeIndex: 4874 entries, 0 to 4873
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        4874 non-null   str
1   Open        4874 non-null   float64
2   High        4874 non-null   float64
3   Low         4874 non-null   float64
4   Close       4874 non-null   float64
5   Adj Close   4874 non-null   float64
6   Volume      4874 non-null   int64
7   new_date    4874 non-null   datetime64[us]
dtypes: datetime64[us](1), float64(5), int64(1), str(1)
memory usage: 304.8 KB
```

```
In [141... stock.Date[0]
stock.new_date[0]
type(stock.new_date[0])
```

Out[141... '2002-05-23'

Out[141... Timestamp('2002-05-23 00:00:00')

Out[141... pandas.Timestamp

시계열값(Timestamp)을 인덱스로 설정

- 인덱스 및 슬라이싱하기 편리함

In [142...

stock.set_index('new_date', inplace=True)
stock.head()

Out[142...

	Date	Open	High	Low	Close	Adj Close	Volume
new_date							
2002-05-23	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000
2002-05-24	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800
2002-05-28	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400
2002-05-29	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800
2002-05-30	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200

In [143...

stock.drop('Date', axis=1, inplace=True)
stock.head()

Out[143...

	Open	High	Low	Close	Adj Close	Volume
new_date						
2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000
2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800
2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400
2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800
2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200

In [144...

stock.index

Out[144...

DatetimeIndex(['2002-05-23', '2002-05-24', '2002-05-28', '2002-05-29',
 '2002-05-30', '2002-05-31', '2002-06-03', '2002-06-04',
 '2002-06-05', '2002-06-06',
 ...
 '2021-09-17', '2021-09-20', '2021-09-21', '2021-09-22',
 '2021-09-23', '2021-09-24', '2021-09-27', '2021-09-28',
 '2021-09-29', '2021-09-30'],
 dtype='datetime64[us]', name='new_date', length=4874, freq=None)

- 2018년 6월 데이터 추출

In [145...

stock.loc['2018-06']

Out [145...

	Open	High	Low	Close	Adj Close	Volume
new_date						
2018-06-01	353.880005	359.989990	352.820007	359.929993	359.929993	7112300
2018-06-04	362.679993	363.000000	355.510010	361.809998	361.809998	7682000
2018-06-05	363.320007	369.829987	361.410004	365.799988	365.799988	8358000
2018-06-06	367.779999	369.679993	363.329987	367.450012	367.450012	7712300
2018-06-07	368.540009	368.700012	357.799988	361.399994	361.399994	8278000
2018-06-08	358.059998	362.390015	356.250000	360.570007	360.570007	5225700
2018-06-11	361.880005	365.670013	360.910004	361.450012	361.450012	4432400
2018-06-12	363.600006	365.980011	362.000000	363.829987	363.829987	4291000
2018-06-13	367.529999	384.250000	364.109985	379.929993	379.929993	18222800
2018-06-14	384.269989	395.029999	383.250000	392.869995	392.869995	14598300
2018-06-15	390.709991	398.859985	387.510010	391.980011	391.980011	13588100
2018-06-18	387.720001	393.160004	386.500000	390.399994	390.399994	6824800
2018-06-19	389.500000	405.290009	388.500000	404.980011	404.980011	16697100
2018-06-20	415.149994	419.470001	409.600006	416.760010	416.760010	16494600
2018-06-21	421.380005	423.209991	406.369995	415.440002	415.440002	18389900
2018-06-22	419.980011	420.500000	409.649994	411.089996	411.089996	10428600
2018-06-25	404.690002	405.989990	378.750000	384.480011	384.480011	22490900
2018-06-26	393.279999	404.779999	389.049988	399.390015	399.390015	15191200
2018-06-27	407.559998	411.589996	390.000000	390.390015	390.390015	16541400
2018-06-28	395.000000	396.899994	387.100006	395.420013	395.420013	12219900
2018-06-29	399.190002	401.329987	390.549988	391.429993	391.429993	9252500

- 2018년 6월~7월 데이터 추출

In [146...

```
stock.loc['2018-06':'2018-07']
```


	Open	High	Low	Close	Adj Close	Volume
new_date						
2018-06-01	353.880005	359.989990	352.820007	359.929993	359.929993	7112300
2018-06-04	362.679993	363.000000	355.510010	361.809998	361.809998	7682000
2018-06-05	363.320007	369.829987	361.410004	365.799988	365.799988	8358000
2018-06-06	367.779999	369.679993	363.329987	367.450012	367.450012	7712300
2018-06-07	368.540009	368.700012	357.799988	361.399994	361.399994	8278000
2018-06-08	358.059998	362.390015	356.250000	360.570007	360.570007	5225700
2018-06-11	361.880005	365.670013	360.910004	361.450012	361.450012	4432400
2018-06-12	363.600006	365.980011	362.000000	363.829987	363.829987	4291000
2018-06-13	367.529999	384.250000	364.109985	379.929993	379.929993	18222800
2018-06-14	384.269989	395.029999	383.250000	392.869995	392.869995	14598300
2018-06-15	390.709991	398.859985	387.510010	391.980011	391.980011	13588100
2018-06-18	387.720001	393.160004	386.500000	390.399994	390.399994	6824800
2018-06-19	389.500000	405.290009	388.500000	404.980011	404.980011	16697100
2018-06-20	415.149994	419.470001	409.600006	416.760010	416.760010	16494600
2018-06-21	421.380005	423.209991	406.369995	415.440002	415.440002	18389900
2018-06-22	419.980011	420.500000	409.649994	411.089996	411.089996	10428600
2018-06-25	404.690002	405.989990	378.750000	384.480011	384.480011	22490900
2018-06-26	393.279999	404.779999	389.049988	399.390015	399.390015	15191200
2018-06-27	407.559998	411.589996	390.000000	390.390015	390.390015	16541400
2018-06-28	395.000000	396.899994	387.100006	395.420013	395.420013	12219900
2018-06-29	399.190002	401.329987	390.549988	391.429993	391.429993	9252500
2018-07-02	385.450012	398.380005	380.000000	398.179993	398.179993	8142500
2018-07-03	399.489990	399.980011	389.500000	390.519989	390.519989	5280300
2018-07-05	393.799988	399.239990	390.859985	398.390015	398.390015	8448900
2018-07-06	397.450012	408.649994	395.519989	408.250000	408.250000	8629600
2018-07-09	415.950012	419.119995	411.100006	418.970001	418.970001	11127500
2018-07-10	417.239990	419.440002	413.079987	415.630005	415.630005	9382900
2018-07-11	411.339996	419.769989	410.600006	418.649994	418.649994	9713900
2018-07-12	415.160004	416.790009	407.799988	413.500000	413.500000	12743300
2018-07-13	409.190002	410.000000	395.079987	395.799988	395.799988	15719000
2018-07-16	398.980011	403.359985	391.750000	400.480011	400.480011	22960000
2018-07-17	346.950012	385.000000	344.000000	379.480011	379.480011	58410400
2018-07-18	381.239990	383.130005	372.359985	375.130005	375.130005	21746300
2018-07-19	371.059998	375.750000	363.000000	364.230011	364.230011	16878700
2018-07-20	364.920013	370.500000	360.140015	361.049988	361.049988	15096700

	Open	High	Low	Close	Adj Close	Volume
new_date						
2018-07-23	359.149994	363.899994	353.600006	362.660004	362.660004	11505200
2018-07-24	366.940002	367.399994	354.559998	357.320007	357.320007	12851500
2018-07-25	357.570007	363.279999	355.649994	362.869995	362.869995	8467800
2018-07-26	358.190002	365.540009	356.630005	363.089996	363.089996	6993700
2018-07-27	366.850006	367.000000	351.649994	355.209991	355.209991	8949500
2018-07-30	351.929993	352.029999	334.019989	334.959991	334.959991	18260700
2018-07-31	331.510010	342.500000	328.000000	337.450012	337.450012	14085400

- 2018년 6월25일 ~7월10일 데이터 추출

In [147...

```
stock.loc['2018-06-25':'2018-07-10']
```

Out[147...

	Open	High	Low	Close	Adj Close	Volume
new_date						
2018-06-25	404.690002	405.989990	378.750000	384.480011	384.480011	22490900
2018-06-26	393.279999	404.779999	389.049988	399.390015	399.390015	15191200
2018-06-27	407.559998	411.589996	390.000000	390.390015	390.390015	16541400
2018-06-28	395.000000	396.899994	387.100006	395.420013	395.420013	12219900
2018-06-29	399.190002	401.329987	390.549988	391.429993	391.429993	9252500
2018-07-02	385.450012	398.380005	380.000000	398.179993	398.179993	8142500
2018-07-03	399.489990	399.980011	389.500000	390.519989	390.519989	5280300
2018-07-05	393.799988	399.239990	390.859985	398.390015	398.390015	8448900
2018-07-06	397.450012	408.649994	395.519989	408.250000	408.250000	8629600
2018-07-09	415.950012	419.119995	411.100006	418.970001	418.970001	11127500
2018-07-10	417.239990	419.440002	413.079987	415.630005	415.630005	9382900

② Timestamp를 Period로 변환 : Timestamp.to_period(freq='D'|'M'|'Y')

- Timestamp 객체를 일정한 기간을 나타내는 Period 객체로 변환
- DatetimeIndex가 PeriodIndex로 변환
- 자료형은 datetime64에서 period로 변환
- freq 옵션 : https://pandas.pydata.org/docs/user_guide/timeseries.html#timeseries-period-aliases

옵션	설명	옵션	설명
D	day(1일)	B	business day(휴일 제외)
W	week(1주)	H	hour(1시간)
M	month end(월말)	T	minute(1분)
MS	month begin(월초)	S	second(1초)
Q	quarter end(분기말)	L	millisecond(1/1,000초)
QS	quarter begin(분기초)	U	microsecond(1/1,000,000초)
A	year end(연말)	N	nanosecond(1/1,000,000,000초)
AS	year begin(연초)	...	...

```
In [149... dates = ['2019-01-01', '2020-03-01', '2012-06-01']
ts_dates = pd.to_datetime(dates)
ts_dates
```

```
Out[149... DatetimeIndex(['2019-01-01', '2020-03-01', '2012-06-01'], dtype='datetime64[us]', freq=None)
```

```
In [150... ts_dates.to_period(freq='D')
```

```
Out[150... PeriodIndex(['2019-01-01', '2020-03-01', '2012-06-01'], dtype='period[D]')
```

```
In [151... ts_dates.to_period(freq='M')
```

```
Out[151... PeriodIndex(['2019-01', '2020-03', '2012-06'], dtype='period[M]')
```

```
In [152... ts_dates.to_period(freq='Y')
```

```
Out[152... PeriodIndex(['2019', '2020', '2012'], dtype='period[Y-DEC]')
```

```
In [154... ts_dates.to_period(freq='h')
```

```
Out[154... PeriodIndex(['2019-01-01 00:00', '2020-03-01 00:00', '2012-06-01 00:00'], dtype='period[h]')
```

2) 시계열 데이터 생성

Timestamp 배열 생성 : date_range() 함수 사용

- date_range(start=None, end=None, periods=None, freq=None, tz=None, normalize=False, name=None, closed=_NoDefault.no_default, inclusive=None, **kwargs)
- DatetimeIndex 생성
- https://pandas.pydata.org/docs/reference/api/pandas.date_range.html

```
In [155... # 월초기준 : M
pd.date_range(start='2020-01-01', periods=6, freq='MS', tz='Asia/Seoul')
```

```
Out[155... DatetimeIndex(['2020-01-01 00:00:00+09:00', '2020-02-01 00:00:00+09:00',
                '2020-03-01 00:00:00+09:00', '2020-04-01 00:00:00+09:00',
                '2020-05-01 00:00:00+09:00', '2020-06-01 00:00:00+09:00'],
                dtype='datetime64[us, Asia/Seoul]', freq='MS')
```

```
In [156... # 월말기준 : ME
pd.date_range(start='2020-01-01', periods=6, freq='ME', tz='Asia/Seoul')
```

```
Out[156... DatetimeIndex(['2020-01-31 00:00:00+09:00', '2020-02-29 00:00:00+09:00',  
                '2020-03-31 00:00:00+09:00', '2020-04-30 00:00:00+09:00',  
                '2020-05-31 00:00:00+09:00', '2020-06-30 00:00:00+09:00'],  
                dtype='datetime64[us, Asia/Seoul]', freq='ME')
```

```
In [158... pd.date_range(start='2020-01-01', periods=6, freq='3D', tz='Asia/Seoul')
```

```
Out[158... DatetimeIndex(['2020-01-01 00:00:00+09:00', '2020-01-04 00:00:00+09:00',  
                '2020-01-07 00:00:00+09:00', '2020-01-10 00:00:00+09:00',  
                '2020-01-13 00:00:00+09:00', '2020-01-16 00:00:00+09:00'],  
                dtype='datetime64[us, Asia/Seoul]', freq='3D')
```

```
In [159... pd.date_range(start='2020-01-01', periods=6, freq='10h', tz='Asia/Seoul')
```

```
Out[159... DatetimeIndex(['2020-01-01 00:00:00+09:00', '2020-01-01 10:00:00+09:00',  
                '2020-01-01 20:00:00+09:00', '2020-01-02 06:00:00+09:00',  
                '2020-01-02 16:00:00+09:00', '2020-01-03 02:00:00+09:00'],  
                dtype='datetime64[us, Asia/Seoul]', freq='10h')
```

Period 배열 생성 : period_range() 함수 사용

- period_range(start=None, end=None, periods=None, freq=None, name=None)
- PeriodIndex 생성
- https://pandas.pydata.org/docs/reference/api/pandas.period_range.html

```
In [160... pd.period_range(start='2020-01-01', periods=6, freq='M')
```

```
Out[160... PeriodIndex(['2020-01', '2020-02', '2020-03', '2020-04', '2020-05', '2020-06'], dtype='period  
[M]')
```

```
In [161... pd.period_range(start='2020-01-01', periods=6, freq='D')
```

```
Out[161... PeriodIndex(['2020-01-01', '2020-01-02', '2020-01-03', '2020-01-04',  
                '2020-01-05', '2020-01-06'],  
                dtype='period[D]')
```

```
In [162... pd.period_range(start='2020-01-01', periods=6, freq='h')
```

```
Out[162... PeriodIndex(['2020-01-01 00:00', '2020-01-01 01:00', '2020-01-01 02:00',  
                '2020-01-01 03:00', '2020-01-01 04:00', '2020-01-01 05:00'],  
                dtype='period[h]')
```

```
In [163... pd.period_range(start='2020-01-01', periods=6, freq='3h')
```

```
Out[163... PeriodIndex(['2020-01-01 00:00', '2020-01-01 03:00', '2020-01-01 06:00',  
                '2020-01-01 09:00', '2020-01-01 12:00', '2020-01-01 15:00'],  
                dtype='period[3h]')
```

```
In [164... pd.period_range(start='2020-01-01', periods=6, freq='s')
```

```
Out[164... PeriodIndex(['2020-01-01 00:00:00', '2020-01-01 00:00:01',  
                '2020-01-01 00:00:02', '2020-01-01 00:00:03',  
                '2020-01-01 00:00:04', '2020-01-01 00:00:05'],  
                dtype='period[s]')
```

```
In [165... pd.period_range(start='2020-01-01', periods=6, freq='30s')
```

```
Out[165... PeriodIndex(['2020-01-01 00:00:00', '2020-01-01 00:00:30',  
                '2020-01-01 00:01:00', '2020-01-01 00:01:30',  
                '2020-01-01 00:02:00', '2020-01-01 00:02:30'],  
                dtype='period[30s]')
```

3) 시계열 데이터 활용

날짜 데이터 분리

- 연-월-일 정보를 연, 월, 일 각각으로 구분

In [166...

```
stock.head()
stock.info()
```

Out[166...

	Open	High	Low	Close	Adj Close	Volume
new_date						
2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000
2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800
2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400
2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800
2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200

```
<class 'pandas.DataFrame'>
DatetimeIndex: 4874 entries, 2002-05-23 to 2021-09-30
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Open        4874 non-null   float64
1   High        4874 non-null   float64
2   Low         4874 non-null   float64
3   Close       4874 non-null   float64
4   Adj Close   4874 non-null   float64
5   Volume      4874 non-null   int64
dtypes: float64(5), int64(1)
memory usage: 395.6 KB
```

In [167...

```
stock = stock.reset_index()
stock.head()
stock.info()
```

Out[167...

	new_date	Open	High	Low	Close	Adj Close	Volume
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200

```
<class 'pandas.DataFrame'>
RangeIndex: 4874 entries, 0 to 4873
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   new_date    4874 non-null   datetime64[us]
1   Open        4874 non-null   float64
2   High        4874 non-null   float64
3   Low         4874 non-null   float64
4   Close       4874 non-null   float64
5   Adj Close   4874 non-null   float64
6   Volume      4874 non-null   int64
dtypes: datetime64[us](1), float64(5), int64(1)
memory usage: 266.7 KB
```

- **Timestamp 객체**는 연, 월, 일, 시, 분, 초로 추출가능
 - Series.dt.year
 - Series.dt.month
 - Series.dt.day
 - Series.dt.hour
 - Series.dt.minute
 - Series.dt.second

In [169...

```
stock.new_date.dtype
type(stock.new_date)
```

Out[169...

```
dtype('<M8[us]')
```

Out[169...

```
pandas.Series
```

In [171...

```
stock['Year'] = stock.new_date.dt.year
stock['Month'] = stock.new_date.dt.month
stock['day'] = stock.new_date.dt.day
stock.head()
```

Out[171...

	new_date	Open	High	Low	Close	Adj Close	Volume	Year	Month	day
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	5	23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	5	24
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	5	28
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	5	29
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	5	30

- **Period 객체**에서 연-월 또는 연도 추출

In [172...

```
stock['date_year'] = stock.new_date.dt.to_period(freq='Y')
stock['date_month'] = stock.new_date.dt.to_period(freq='M')
stock['date_day'] = stock.new_date.dt.to_period(freq='D')
stock.head()
stock.info()
```

Out[172...

	new_date	Open	High	Low	Close	Adj Close	Volume	Year	Month	day	date_
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	5	23	2
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	5	24	2
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	5	28	2
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	5	29	2
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	5	30	2

```
<class 'pandas.DataFrame'>
RangeIndex: 4874 entries, 0 to 4873
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  -
0   new_date    4874 non-null   datetime64[us]
1   Open        4874 non-null   float64
2   High        4874 non-null   float64
3   Low         4874 non-null   float64
4   Close       4874 non-null   float64
5   Adj Close   4874 non-null   float64
6   Volume      4874 non-null   int64
7   Year        4874 non-null   int32
8   Month       4874 non-null   int32
9   day         4874 non-null   int32
10  date_year   4874 non-null   period[Y-DEC]
11  date_month  4874 non-null   period[M]
12  date_day    4874 non-null   period[D]
dtypes: datetime64[us](1), float64(5), int32(3), int64(1), period[D](1), period[M](1), period
[Y-DEC](1)
memory usage: 438.0 KB
```

날짜 인덱스 활용

```
In [173... stock.set_index('new_date', inplace=True)
stock.head()
```

Out[173...

	Open	High	Low	Close	Adj Close	Volume	Year	Month	day	date_year
new_date										
2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	5	23	2002
2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	5	24	2002
2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	5	28	2002
2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	5	29	2002
2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	5	30	2002

```
In [174... stock.loc['2018'][:5]
```

Out[174...

	Open	High	Low	Close	Adj Close	Volume	Year	Month	day
new_date									
2018-01-02	196.100006	201.649994	195.419998	201.070007	201.070007	10966900	2018	1	2
2018-01-03	202.050003	206.210007	201.500000	205.050003	205.050003	8591400	2018	1	3
2018-01-04	206.199997	207.050003	204.000000	205.630005	205.630005	6029600	2018	1	4
2018-01-05	207.250000	210.020004	205.589996	209.990005	209.990005	7033200	2018	1	5
2018-01-08	210.020004	212.500000	208.440002	212.050003	212.050003	5580200	2018	1	8

In [175...

stock.loc['2018-12'][:3]

Out[175...

	Open	High	Low	Close	Adj Close	Volume	Year	Month	day
new_date									
2018-12-03	293.190002	298.720001	284.579987	290.299988	290.299988	14117400	2018	12	3
2018-12-04	288.130005	295.739990	274.720001	275.329987	275.329987	12800600	2018	12	4
2018-12-06	268.329987	283.220001	267.140015	282.880005	282.880005	13074300	2018	12	6

In [176...

stock.loc['2018-06-20':'2018-06-25':2]

Out[176...

	Open	High	Low	Close	Adj Close	Volume	Year	Month	day
new_date									
2018-06-20	415.149994	419.470001	409.600006	416.760010	416.760010	16494600	2018	6	20
2018-06-22	419.980011	420.500000	409.649994	411.089996	411.089996	10428600	2018	6	22

In [178...

stock.index
stock.index.max()

Out[178...

DatetimeIndex(['2002-05-23', '2002-05-24', '2002-05-28', '2002-05-29',
 '2002-05-30', '2002-05-31', '2002-06-03', '2002-06-04',
 '2002-06-05', '2002-06-06',
 ...,
 '2021-09-17', '2021-09-20', '2021-09-21', '2021-09-22',
 '2021-09-23', '2021-09-24', '2021-09-27', '2021-09-28',
 '2021-09-29', '2021-09-30'],
 dtype='datetime64[us]', name='new_date', length=4874, freq=None)

Out[178...

Timestamp('2021-09-30 00:00:00')

In [179...

기준일자를 기반으로 일수를 인덱스로
recency = stock.index.max()
stock['time_delta'] = recency - stock.index
stock.info()

<class 'pandas.DataFrame'>
DatetimeIndex: 4874 entries, 2002-05-23 to 2021-09-30
Data columns (total 13 columns):
Column Non-Null Count Dtype
--- -
0 Open 4874 non-null float64
1 High 4874 non-null float64
2 Low 4874 non-null float64
3 Close 4874 non-null float64
4 Adj Close 4874 non-null float64
5 Volume 4874 non-null int64
6 Year 4874 non-null int32
7 Month 4874 non-null int32
8 day 4874 non-null int32
9 date_year 4874 non-null period[Y-DEC]
10 date_month 4874 non-null period[M]
11 date_day 4874 non-null period[D]
12 time_delta 4874 non-null timedelta64[us]
dtypes: float64(5), int32(3), int64(1), period[D](1), period[M](1), period[Y-DEC](1), timedelta64[us](1)
memory usage: 605.0 KB

In [180...

stock.head()

Out[180...

	Open	High	Low	Close	Adj Close	Volume	Year	Month	day	date_year
new_date										
2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	5	23	2002
2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	5	24	2002
2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	5	28	2002
2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	5	29	2002
2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	5	30	2002

In [181...

stock.set_index('time_delta', inplace=True)
stock.head(3)

Out[181...

	Open	High	Low	Close	Adj Close	Volume	Year	Month	day	date_yea
time_delta										
7070 days	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	5	23	200
7069 days	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	5	24	200
7065 days	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	5	28	200

In [182...

```
stock['200 days':'180 day']
```

Out[182...

	Open	High	Low	Close	Adj Close	Volume	Year	Month	day
time_delta									
199 days	516.320007	520.729980	508.029999	520.250000	520.250000	3484300	2021	3	15
198 days	524.469971	533.419983	523.210022	524.030029	524.030029	3238900	2021	3	16
197 days	522.000000	528.369995	514.299988	524.440002	524.440002	2817400	2021	3	17
196 days	516.400024	517.900024	503.850006	504.790009	504.790009	4094500	2021	3	18
195 days	504.959991	513.190002	501.630005	512.179993	512.179993	4386400	2021	3	19
192 days	509.130005	529.309998	509.130005	523.109985	523.109985	3360600	2021	3	22
191 days	529.869995	543.119995	529.400024	535.090027	535.090027	5583500	2021	3	23
190 days	533.780029	534.419983	520.000000	520.809998	520.809998	4102200	2021	3	24
189 days	516.989990	518.530029	497.000000	502.859985	502.859985	4926800	2021	3	25
188 days	502.820007	508.720001	496.679993	508.049988	508.049988	3467800	2021	3	26
185 days	505.660004	518.000000	504.399994	513.950012	513.950012	3330000	2021	3	29
184 days	510.510010	514.409973	506.910004	513.390015	513.390015	2418100	2021	3	30
183 days	515.669983	528.130005	515.440002	521.659973	521.659973	3503100	2021	3	31
182 days	529.929993	540.500000	527.030029	539.419983	539.419983	3938600	2021	4	1

In [183...

```
stock.sort_index()
```

Out[183...

		Open	High	Low	Close	Adj Close	Volume	Year	Month	da
time_delta										
	0 days	608.049988	619.000000	608.049988	610.340027	610.340027	6612600	2021	9	3
	1 days	589.010010	609.880005	588.010010	599.059998	599.059998	6221000	2021	9	2
	2 days	589.000000	599.539978	580.159973	583.849976	583.849976	4431100	2021	9	2
	3 days	587.950012	593.580017	576.929993	592.640015	592.640015	2504700	2021	9	2
	6 days	592.500000	592.979980	583.640015	592.390015	592.390015	2124800	2021	9	2
	...	...	...	...	...	...	...	...	...	...
	7063 days	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	5	3
	7064 days	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	5	2
	7065 days	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	5	2
	7069 days	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	5	2
	7070 days	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	5	2

4874 rows × 12 columns

참고. 날짜 문자열에서 str객체를 이용한 연,월,일 추출

In [184...

stock = pd.read_csv('data/NFLX.csv')
stock.head(2)

Out[184...

	Date	Open	High	Low	Close	Adj Close	Volume
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800

In [185...

stock.Date.dtype

Out[185...

<StringDtype(storage='python', na_value=nan)>

- Series.str.split('-', expand=True)를 이용하여 연,월,일 컬럼으로 저장

In [187...

stock[['Year', 'Month', 'Day']] = stock.Date.str.split('-', expand=True)
stock.head(2)

Out[187... 0 [2002, 05, 23]
1 [2002, 05, 24]
2 [2002, 05, 28]
3 [2002, 05, 29]
4 [2002, 05, 30]

...
4869 [2021, 09, 24]
4870 [2021, 09, 27]
4871 [2021, 09, 28]
4872 [2021, 09, 29]
4873 [2021, 09, 30]
Name: Date, Length: 4874, dtype: object

Out[187...

	Date	Open	High	Low	Close	Adj Close	Volume	Year	Month	Day
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	05	23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	05	24

- Series.str.split('-')로 분할된 컬럼에서 인덱스로 해당위치 값들만 추출

In [188... stock.Date.str.split('-')

Out[188... 0 [2002, 05, 23]
1 [2002, 05, 24]
2 [2002, 05, 28]
3 [2002, 05, 29]
4 [2002, 05, 30]

...
4869 [2021, 09, 24]
4870 [2021, 09, 27]
4871 [2021, 09, 28]
4872 [2021, 09, 29]
4873 [2021, 09, 30]
Name: Date, Length: 4874, dtype: object

In [189... stock.Date.str.split('-').str[0]

Out[189... 0 2002
1 2002
2 2002
3 2002
4 2002

...
4869 2021
4870 2021
4871 2021
4872 2021
4873 2021
Name: Date, Length: 4874, dtype: object

In [190... stock.Date.str.split('-').str[-1]

```
Out[190...] 0      23
            1      24
            2      28
            3      29
            4      30
            ..
          4869    24
          4870    27
          4871    28
          4872    29
          4873    30
Name: Date, Length: 4874, dtype: object
```

- Series.str.split('-')으로 분할한 후 tolist()함수를 적용하여 리스트로 변환한 후 새로운 데이터프레임으로 생성

```
In [194...] stock.Date.str.split('-').tolist()[ :3]
pd.DataFrame(stock.Date.str.split('-').tolist(), columns=['Year', 'Month', 'Day']).head()
```

```
Out[194...] [['2002', '05', '23'], ['2002', '05', '24'], ['2002', '05', '28']]
```

```
Out[194...]   Year  Month  Day
0  2002     05   23
1  2002     05   24
2  2002     05   28
3  2002     05   29
4  2002     05   30
```

- Series.str.split('-')으로 분할한 결과에 apply(pd.Series)를 적용하여 데이터프레임을 생성

```
In [195...] stock.Date.str.split('-').apply(pd.Series)
```

```
Out[195...]   0  1  2
0  2002  05  23
1  2002  05  24
2  2002  05  28
3  2002  05  29
4  2002  05  30
...  ...  ...  ...
4869 2021  09  24
4870 2021  09  27
4871 2021  09  28
4872 2021  09  29
4873 2021  09  30
```

4874 rows × 3 columns

```
In [198... new_df = stock.Date.str.split('-').apply(pd.Series)
new_df.columns = ['Year2', 'Month2', 'Day2']
new_df.index = stock.index
new_df.head()
```

Out[198...

	Year2	Month2	Day2
0	2002	05	23
1	2002	05	24
2	2002	05	28
3	2002	05	29
4	2002	05	30

```
In [200... stock.join(new_df)
stock.head(3)
```

Out[200...

	Date	Open	High	Low	Close	Adj Close	Volume	Year	Month
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	05
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	05
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	05
3	2002-05-29	1.164286	1.164286	1.085714	1.103571	1.103571	6757800	2002	05
4	2002-05-30	1.107857	1.107857	1.071429	1.071429	1.071429	10154200	2002	05
...	...	...	...	...	...	...	...	...	...
4869	2021-09-24	592.500000	592.979980	583.640015	592.390015	592.390015	2124800	2021	09
4870	2021-09-27	587.950012	593.580017	576.929993	592.640015	592.640015	2504700	2021	09
4871	2021-09-28	589.000000	599.539978	580.159973	583.849976	583.849976	4431100	2021	09
4872	2021-09-29	589.010010	609.880005	588.010010	599.059998	599.059998	6221000	2021	09
4873	2021-09-30	608.049988	619.000000	608.049988	610.340027	610.340027	6612600	2021	09

4874 rows × 13 columns

Out[200...

	Date	Open	High	Low	Close	Adj Close	Volume	Year	Month	Day
0	2002-05-23	1.156429	1.242857	1.145714	1.196429	1.196429	104790000	2002	05	23
1	2002-05-24	1.214286	1.225000	1.197143	1.210000	1.210000	11104800	2002	05	24
2	2002-05-28	1.213571	1.232143	1.157143	1.157143	1.157143	6609400	2002	05	28

참고. 시계열 데이터 전처리 4단계

1. resample : 시간 정렬

- 원시 시계열 데이터는
 - 불규칙 샘플링(예. 센서 이벤트 기반)
 - 누락 시간 존재
 - 데이터가 너무 촘촘(초단위)하거나 너무 듬성(일단위)
- 분석 단위로 시간축을 표준화 : 예. 1분 / 10분 / 1시간 / 1일
- 시간단위로 집계 : 다운 샘플링 -> `Series.resample('10min')`
- 누락 처리 : 빈 시간 채우기 -> 평균으로 채우거나 앞뒤값으로 `ffill()` / 보간법 -> `interpolate()`

2. smoothing : 잡음 제거/노이즈 필터링

- 노이즈(noise)는 단기 변동 키워서 추세/계절성 파악 방해, 이상탐지 방해, 모델학습 불안정
- 추세를 살리고, 변동은 줄이는 처리
- 이동평균(Moving Average: Rolling Mean) : `rolling()`
- 지수이동평균(EWMA) : `ewm()`
- 중앙값 필터 : 이상치/스파이크에 강함

3. differencing : 추세 제거 / 정상성 강화

- 시계열 모델이나 패턴분석에서 추세(trend)가 강하면
 - 평균/분산이 시간에 따라 변함(정상성)
 - 상관구조 해석이 어려움
 - 변동(증감) 자체가 더 중요한 경우가 많음
- 차분(differencing)으로 변화량을 분석 : `diff()`
 - 1차 차분
 - 계절 차분
 - 로그변환 + 차분(스케일 안정화)
 - 비율변화 : `pct_change()`

4. binning : 구간화 / 패턴 분석을 위한 범주화

- 연속형 값을 레벨/상태로 바꾸면
 - 패턴이 훨씬 해석 쉬움(예. 낮음/보통/높음)
 - 상태 기반 규칙/알람
 - 마코프 전이(상태 전이) 분석 가능
 - `cut()`, `qcut()`, `value_counts()`
-